

WEBRUM-99: Best Practice Summary

Series: WEBRUM | Notebook: 99 | Created: March 2026 | Last Updated: 03/26/2026

Overview

This notebook consolidates every actionable best practice from the WEBRUM series (notebooks 01-08) into a single definitive reference. Each practice specifies the exact setting, value, or action to take — no hedging, no "it depends." Practices are organized by category and prioritized as Critical, Recommended, or Optional.

Table of Contents

- 1. [RUM Agent Configuration](#)
- 2. [SPA Instrumentation](#)
- 3. [Core Web Vitals Thresholds](#)
- 4. [Performance Optimization](#)
- 5. [Error Monitoring](#)
- 6. [Session Replay and Privacy](#)
- 7. [Session Analysis](#)
- 8. [Dashboards and Alerting](#)
- 9. [DQL Query Patterns](#)

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
RUM Enabled	At least one web application with RUM injection active
Permissions	storage:events:read , storage:metrics:read , storage:entities:read
Previous Notebooks	WEBRUM-01 through WEBRUM-08 (for full context)

1. RUM Agent Configuration

#	Best Practice	Setting / Value	Priority	Source
1	Use automatic injection	Injection method: Automatic (OneAgent modifies HTML responses at the web server)	Critical	WEBRUM-01
2	Filter out bots from all RUM queries	Always apply <code>filter user.type == "REAL_USER"</code> in every DQL query against <code>dt.rum.user_session</code>	Critical	WEBRUM-01
3	Use manual injection for CDN/static-hosted apps	Add RUM <code><script></code> tag to <code><head></code> before all other scripts when no server-side OneAgent exists	Recommended	WEBRUM-02
4	Use agentless RUM only as last resort	Select agentless monitoring only when OneAgent cannot be deployed at all	Optional	WEBRUM-02
5	Set session timeout to 30 minutes	Keep the default 30-minute inactivity timeout for session boundaries	Recommended	WEBRUM-01
6	Enable Fetch API monitoring	Settings > Web and mobile monitoring > Async web requests and SPAs > Fetch requests: Enabled	Critical	WEBRUM-02
7	Deploy RUM + Synthetic together	Use synthetic for availability SLAs and baselines; use RUM for real user experience	Critical	WEBRUM-01, WEBRUM-08

2. SPA Instrumentation

#	Best Practice	Setting / Value	Priority	Source
8	Enable SPA route change capture	Settings > Web and mobile monitoring > Async web requests and SPAs > SPA route changes: Enabled	Critical	WEBRUM-02
9	Enable Async Web Requests capture	Settings > Web and mobile monitoring > Async web requests and SPAs > Async web requests: Enabled	Critical	WEBRUM-02
10	Configure user action naming rules for dynamic URLs	Replace dynamic segments: <code>/users/{id}</code> becomes <code>/users/{*}</code> via Settings > Web and mobile monitoring > User action naming	Critical	WEBRUM-02
11	Use <code>data-dtname</code> attribute on interactive elements	Add <code>data-dtname="Checkout"</code> to buttons/links for meaningful action names	Recommended	WEBRUM-02
12	Load RUM agent before the SPA router initializes	Place the RUM <code><script></code> tag in <code><head></code> before framework bootstrap scripts	Critical	WEBRUM-02
13	Use <code>dtrum.enterAction()</code> / <code>dtrum.leaveAction()</code> for custom actions	Wrap business-critical interactions in manual RUM actions when auto-detection is insufficient	Recommended	WEBRUM-02
14	Validate SPA instrumentation after setup	Query for <code>RouteChange</code> actions — if none appear, SPA monitoring is misconfigured	Critical	WEBRUM-02
15	Separate mobile WebView and desktop into distinct RUM apps	Create two Dynatrace applications; use application detection rules based on User-Agent string	Recommended	WEBRUM-02
16	Add custom User-Agent suffix for WebView detection	Android: <code>setUserAgentString()</code> / iOS: <code>applicationNameForUserAgent</code> with app name + version	Recommended	WEBRUM-02
17	Enable hybrid WebView monitoring in mobile SDK	Android: <code>hybridWebView { enabled true; domains "your-domain.com" }</code> / iOS: <code>DTXHybridApplication = true</code>	Recommended	WEBRUM-02
18	Never instrument third-party WebViews	Only add your own domains to <code>domains</code> / <code>DTXMonitoredDomains</code> — never OAuth, payment, or external pages	Critical	WEBRUM-02

3. Core Web Vitals Thresholds

#	Best Practice	Setting / Value	Priority	Source
19	Target LCP at or below 2.5 seconds	Largest Contentful Paint p75 <= 2500ms. Poor threshold: > 4000ms	Critical	WEBRUM-03
20	Target INP at or below 200 milliseconds	Interaction to Next Paint p75 <= 200ms. Poor threshold: > 500ms	Critical	WEBRUM-03
21	Target CLS at or below 0.1	Cumulative Layout Shift p75 <= 0.1. Poor threshold: > 0.25	Critical	WEBRUM-03
22	Use p75 (not average) for CWV evaluation	Google uses 75th percentile as the pass/fail threshold for CWV	Critical	WEBRUM-03
23	Track CWV per page, not just per app	Aggregate CWV by <code>action.name</code> to identify which specific pages fail thresholds	Recommended	WEBRUM-03
24	Set explicit <code>width</code> and <code>height</code> on all images	Prevents CLS from images loading without reserved space	Critical	WEBRUM-03
25	Use <code>font-display: swap</code> with fallback font metrics	Prevents CLS from FOUT (Flash of Unstyled Text) during web font loading	Recommended	WEBRUM-03
26	Use fixed dimensions on ads, embeds, and iframes	Set explicit <code>width</code> / <code>height</code> on all third-party content containers to prevent layout shifts	Recommended	WEBRUM-03
27	Break long JavaScript tasks to improve INP	Split tasks > 50ms using <code>requestIdleCallback</code> , <code>setTimeout(0)</code> , or web workers	Critical	WEBRUM-03
28	Batch DOM reads and writes to reduce INP	Avoid forced layout/reflow by separating DOM read and write operations	Recommended	WEBRUM-03
29	Build a CWV scorecard query	Use the single-query pattern from WEBRUM-03 to show % Good/NI/Poor for all three metrics	Recommended	WEBRUM-03
30	Track CWV trends at hourly (LCP/INP) and daily (CLS) granularity	Use <code>makeTimeseries</code> with <code>interval:1h</code> for LCP/INP, <code>interval:1d</code> for CLS	Recommended	WEBRUM-03

4. Performance Optimization

#	Best Practice	Setting / Value	Priority	Source
31	Target TTFB at or below 800 milliseconds	Time to First Byte p75 <= 800ms (Google recommendation)	Critical	WEBRUM-06
32	Use DNS prefetching for third-party domains	Add <code><link rel="dns-prefetch" href="//cdn.example.com"></code> for external resources	Recommended	WEBRUM-06
33	Enable HTTP/2 on all web servers	Reduces TCP connection overhead; multiplexes requests on a single connection	Critical	WEBRUM-06
34	Enable TLS 1.3 with OCSP stapling	Reduces SSL handshake time by one round trip	Recommended	WEBRUM-06
35	Defer non-critical JavaScript	Use <code>defer</code> or <code>async</code> attributes on <code><script></code> tags that are not needed for initial render	Critical	WEBRUM-06
36	Lazy-load below-the-fold images	Use <code>loading="lazy"</code> on <code></code> tags not visible in the initial viewport	Recommended	WEBRUM-06
37	Monitor the DOM Interactive to Load Event gap	A large gap indicates excessive sub-resource loading; target < 2 seconds	Recommended	WEBRUM-06
38	Deploy CDN edge servers for high-TTFB regions	If TTFB is high for specific geographies, add CDN nodes closer to those users	Critical	WEBRUM-06
39	Prioritize slow pages by impact score	Calculate <code>impact_score = page_views * avg_duration_ms</code> and fix highest-impact pages first	Recommended	WEBRUM-06
40	Set page load performance SLA at 3 seconds	Track % of page loads with duration <= 3s as primary performance SLA	Critical	WEBRUM-08

5. Error Monitoring

#	Best Practice	Setting / Value	Priority	Source
41	Target error session rate below 2%	% of sessions with <code>error.count > 0 < 2%</code>	Critical	WEBRUM-05, WEBRUM-08
42	Rank errors by affected session count, not occurrence count	A retry loop in one session inflates occurrence count; <code>countDistinct(session.id)</code> is the true impact metric	Critical	WEBRUM-05
43	Monitor XHR/fetch errors separately from JS errors	Filter <code>error.type == "XHR_ERROR"</code> or <code>error.type == "FETCH_ERROR"</code> to isolate backend API failures	Recommended	WEBRUM-05
44	Enable rage click detection	Dynatrace captures rage clicks automatically; query <code>rage.click == true</code> to find frustrated users	Critical	WEBRUM-05
45	Correlate errors with session outcomes	Compare bounce rate and avg actions for "With Errors" vs "No Errors" sessions to quantify error impact	Recommended	WEBRUM-05
46	Segment error rates by browser family	Query <code>error.count > 0</code> grouped by <code>browser.family</code> to identify browser-specific issues	Recommended	WEBRUM-05
47	Alert on new error messages	Query errors with <code>min(timestamp)</code> in the last hour to detect errors introduced by recent deployments	Critical	WEBRUM-08
48	Use <code>dtrum.reportError()</code> for custom business errors	Report application-specific errors (e.g., payment validation failures) via the RUM API	Recommended	WEBRUM-05

6. Session Replay and Privacy

#	Best Practice	Setting / Value	Priority	Source
49	Set Session Replay default masking to "Mask user input"	Settings > Session Replay > Data privacy > Default masking level: Mask user input	Critical	WEBRUM-07
50	Block entire PII containers with <code>data-dtrum-block="true"</code>	Add <code>data-dtrum-block="true"</code> to any <code><div></code> containing sensitive data (SSN, CC, health info)	Critical	WEBRUM-07
51	Add CSS masking rules for all form inputs with sensitive data	Settings > Session Replay > Data privacy: add selectors like <code>.credit-card-form input</code> , <code>[data-sensitive]</code>	Critical	WEBRUM-07
52	Set replay sampling rate based on traffic volume	High-traffic production: 1-5% ; internal apps: 10-25% ; pre-prod: 50-100%	Recommended	WEBRUM-07
53	Enable error-triggered replay capture	Configure conditional capture: automatically record sessions when JavaScript errors occur	Recommended	WEBRUM-07
54	Enable frustration-triggered replay capture	Configure conditional capture for sessions with rage clicks or exit intent	Recommended	WEBRUM-07
55	Err on over-masking, not under-masking	It is better to mask non-sensitive content than to accidentally capture PII	Critical	WEBRUM-07
56	Conduct a privacy review before enabling Session Replay in production	Review all captured content against GDPR, CCPA, and HIPAA requirements before go-live	Critical	WEBRUM-07
57	Use Dynatrace native replay over third-party tools	Native replay integrates with performance waterfall, backend traces, and Davis AI; third-party tools lack this correlation	Recommended	WEBRUM-07
58	Follow the replay investigation workflow	1) Identify via DQL, 2) Watch replay, 3) Correlate with waterfall, 4) Note pattern, 5) Quantify with DQL	Recommended	WEBRUM-07

7. Session Analysis

#	Best Practice	Setting / Value	Priority	Source
59	Define custom session properties for business context	Configure Settings > Web and mobile monitoring > Session and user action properties with: <code>session.user_id</code> , <code>session.plan_type</code> , <code>session.cart_value</code> , <code>session.ab_variant</code>	Critical	WEBRUM-04
60	Target bounce rate below 40%	% of sessions with action.count == 1 < 40%	Recommended	WEBRUM-04, WEBRUM-08
61	Segment sessions by engagement level	Bucket sessions: Bounce (1 action), Low (2-3), Medium (4-10), High (10+)	Recommended	WEBRUM-04
62	Track entry and exit pages	Use <code>takeFirst(action.name)</code> and <code>takeLast(action.name)</code> grouped by <code>session.id</code> to map user journeys	Recommended	WEBRUM-04
63	Use session properties for conversion tracking, not URL pattern matching	Mark conversions via session properties instead of fragile <code>action.name ~ "*confirmation*"</code> patterns	Critical	WEBRUM-04
64	Analyze sessions by hour of day for capacity planning	Group sessions by <code>getHour(timestamp)</code> over 7 days to identify peak traffic windows	Optional	WEBRUM-04
65	Segment by geography and device	Break down sessions by <code>country</code> , <code>browser.family</code> , <code>os.family</code> , and <code>connection.type</code> to find regional/device issues	Recommended	WEBRUM-04, WEBRUM-06

8. Dashboards and Alerting

#	Best Practice	Setting / Value	Priority	Source
66	Set Apdex threshold T = 3 seconds for page loads	Satisfied: <= 3s, Tolerating: 3-12s, Frustrated: > 12s	Critical	WEBRUM-08
67	Set Apdex threshold T = 2.5 seconds for XHR and route changes	Satisfied: <= 2.5s, Tolerating: 2.5-10s, Frustrated: > 10s	Critical	WEBRUM-08
68	Target Apdex score above 0.85	Apdex 0.85-0.93 = Good; 0.94-1.00 = Excellent; below 0.70 = investigate immediately	Critical	WEBRUM-08
69	Build an executive dashboard with 6 KPIs	Tiles: Apdex, session count, error rate %, bounce rate %, CWV pass rate %, avg session duration	Recommended	WEBRUM-08
70	Build an operational dashboard with real-time error view	Tiles: error count per 15m by type, active error table (last 1h), performance SLA % under 3s	Recommended	WEBRUM-08
71	Alert on error rate spike	Threshold: > 5% of sessions with errors sustained for 15 minutes	Critical	WEBRUM-08
72	Alert on Apdex drop	Threshold: Apdex < 0.7 sustained for 15 minutes	Critical	WEBRUM-08
73	Alert on performance degradation	Threshold: p75 page load > 5 seconds sustained for 15 minutes	Critical	WEBRUM-08
74	Alert on traffic anomaly	Threshold: session count < 50% of previous day's hourly average	Recommended	WEBRUM-08
75	Alert on CWV regression	Threshold: LCP p75 > 4 seconds sustained for 30 minutes	Recommended	WEBRUM-08
76	Use metric event evaluation window of 15 minutes	Settings > Anomaly detection > Metric events : evaluation window 15m , sliding window 5m	Recommended	WEBRUM-08
77	Prefer Davis AI anomaly detection over static thresholds in production	Davis automatically baselines normal behavior, reducing false positives from seasonal patterns	Recommended	WEBRUM-08
78	Combine RUM and synthetic in a single dashboard	Use <code>append</code> to show side-by-side comparison: RUM real-user metrics vs synthetic clean-room metrics	Recommended	WEBRUM-08
79	Track CWV pass rate as executive KPI	Target: > 75% of page loads meeting all three CWV thresholds	Recommended	WEBRUM-08

9. DQL Query Patterns

#	Best Practice	Setting / Value	Priority	Source
80	Always include <code>from:</code> time range on RUM queries	Use <code>from:-1h</code> for real-time, <code>from:-24h</code> for daily, <code>from:-7d</code> for trends	Critical	WEBRUM-01
81	Always filter <code>user.type == "REAL_USER"</code> on session queries	Excludes bots (<code>ROBOT</code>) and synthetic monitors (<code>SYNTHETIC</code>) from real-user analysis	Critical	WEBRUM-01
82	Use <code>isNotNull()</code> before aggregating optional fields	Apply filter <code>isNotNull(web_vitals.largest_contentful_paint)</code> before CWV aggregations	Critical	WEBRUM-03
83	Use <code>countDistinct(session.id)</code> for error impact measurement	Never use <code>count()</code> alone — it inflates impact from retry loops in single sessions	Critical	WEBRUM-05
84	Require minimum sample size before drawing conclusions	Use <code>filter page_views > 20</code> (or similar) to avoid misleading statistics from low-volume pages	Recommended	WEBRUM-03, WEBRUM-06
85	Use nanosecond-to-millisecond conversion: <code>/ 1000000.0</code>	Dynatrace duration fields are in nanoseconds; divide by 1,000,000 for milliseconds, 1,000,000,000 for seconds	Critical	WEBRUM-03
86	Alias all aggregations before using in <code>sort</code>	Always write <code>summarize c = count()</code> then <code>sort c desc</code> , never <code>sort count() desc</code>	Critical	All

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.